

Lucida — code evidence

Esme Moula Chowdhury Abha · 26-94089-2 · sole author

System	LangGraph supervisor + 8 specialists, per-shop SQLite, vector memory
Size	52 Python files · ~21,000 lines · 22 tables · 87 tests
Authorship	81 commits, one contributor, 14 Aug – 2 Sep 2026
Live integrations	Pathao courier (booked a real consignment), Telegram, OpenAI, DuckDuckGo
This document	24 excerpts, each captioned with the marking criterion it evidences

graph.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

- src
 - lucida
 - agents
 - ad_creative.py
 - base.py
 - delivery.py
 - engagement.py
 - inventory.py
 - market_research.py
 - pricing_agent.py
 - product_vision.py
 - reporting.py
 - memory
 - db.py
 - vector.py
 - tools
 - channels.py
 - connections.py
 - courier.py
 - imagegen.py
 - graph.py
 - llm.py
 - supervisor.py
 - tests
 - test_workforce.py
 - web
 - app_ui.py
 - bridge.py
 - serve.py

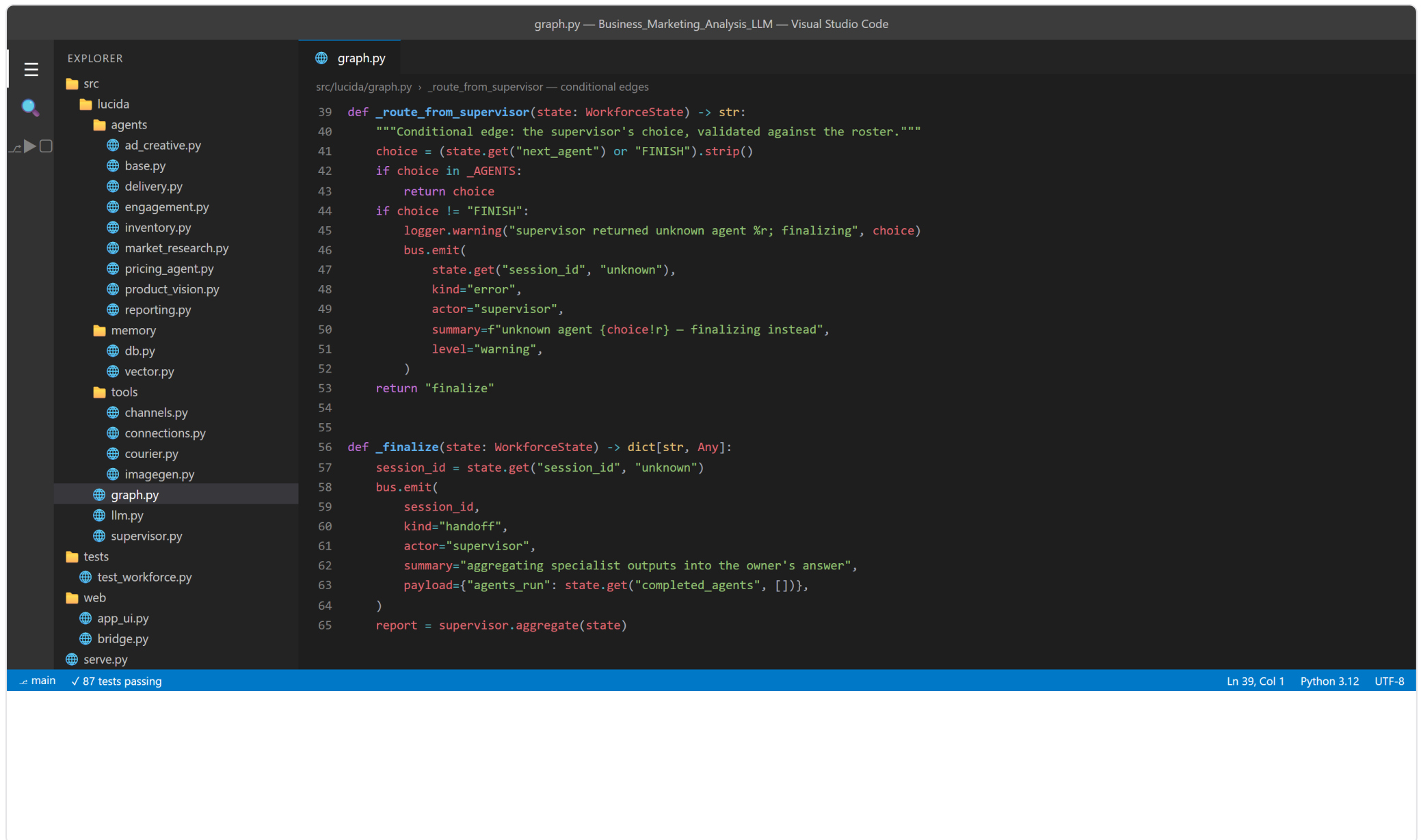
graph.py

```
src/lucida/graph.py > build_graph — the star topology compiled

85 def build_graph(checkpointer=None):
86     """Assemble and compile the state graph."""
87     graph = StateGraph(WorkforceState)
88
89     graph.add_node("supervisor", supervisor)
90     for name, agent in _AGENTS.items():
91         graph.add_node(name, agent)
92     graph.add_node("finalize", _finalize)
93
94     graph.add_edge(START, "supervisor")
95     graph.add_conditional_edges(
96         "supervisor",
97         _route_from_supervisor,
98         **{n: n for n in AGENT_NAMES}, "finalize": "finalize"},
99     )
100     for name in AGENT_NAMES:
101         graph.add_edge(name, "supervisor")
102     graph.add_edge("finalize", END)
103
104     return graph.compile(checkpointer=checkpointer)
```

main ✓ 87 tests passing Ln 85, Col 1 Python 3.12 UTF-8

Every request enters at the supervisor. Specialists return to it and never call each other, so the route taken is a decision, not a fixed pipeline.



graph.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

- src
 - lucida
 - agents
 - ad_creative.py
 - base.py
 - delivery.py
 - engagement.py
 - inventory.py
 - market_research.py
 - pricing_agent.py
 - product_vision.py
 - reporting.py
 - memory
 - db.py
 - vector.py
 - tools
 - channels.py
 - connections.py
 - courier.py
 - imagegen.py
 - graph.py
 - llm.py
 - supervisor.py
 - tests
 - test_workforce.py
 - web
 - app_ui.py
 - bridge.py
 - serve.py

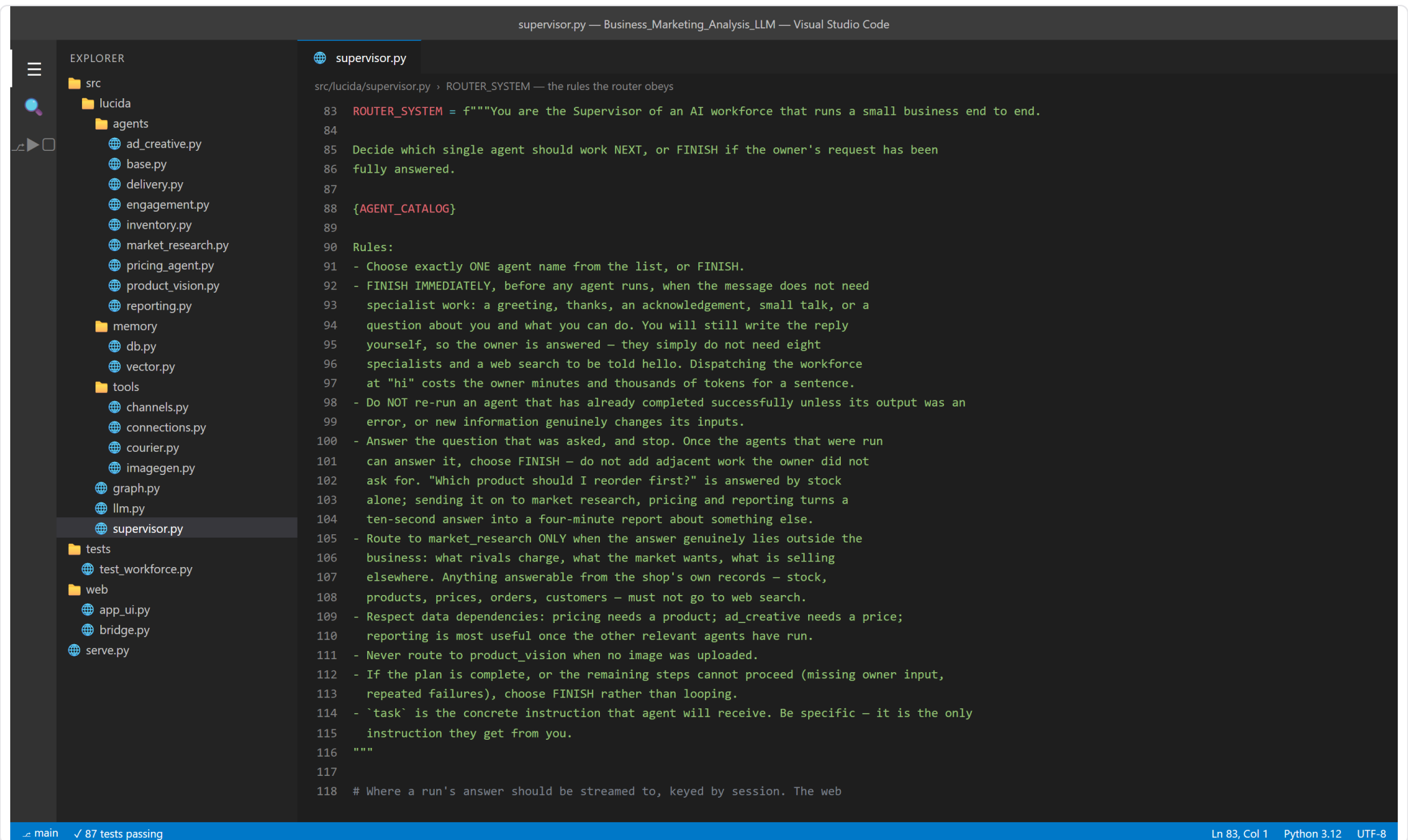
graph.py

```
src/lucida/graph.py > _route_from_supervisor — conditional edges

39 def _route_from_supervisor(state: WorkforceState) -> str:
40     """Conditional edge: the supervisor's choice, validated against the roster."""
41     choice = (state.get("next_agent") or "FINISH").strip()
42     if choice in _AGENTS:
43         return choice
44     if choice != "FINISH":
45         logger.warning("supervisor returned unknown agent %r; finalizing", choice)
46         bus.emit(
47             state.get("session_id", "unknown"),
48             kind="error",
49             actor="supervisor",
50             summary=f"unknown agent {choice!r} - finalizing instead",
51             level="warning",
52         )
53     return "finalize"
54
55
56 def _finalize(state: WorkforceState) -> dict[str, Any]:
57     session_id = state.get("session_id", "unknown")
58     bus.emit(
59         session_id,
60         kind="handoff",
61         actor="supervisor",
62         summary="aggregating specialist outputs into the owner's answer",
63         payload={"agents_run": state.get("completed_agents", [])},
64     )
65     report = supervisor.aggregate(state)
```

main ✓ 87 tests passing Ln 39, Col 1 Python 3.12 UTF-8

The conditional edge that makes the star a star: the supervisor's choice decides the next node, or finishes the run.

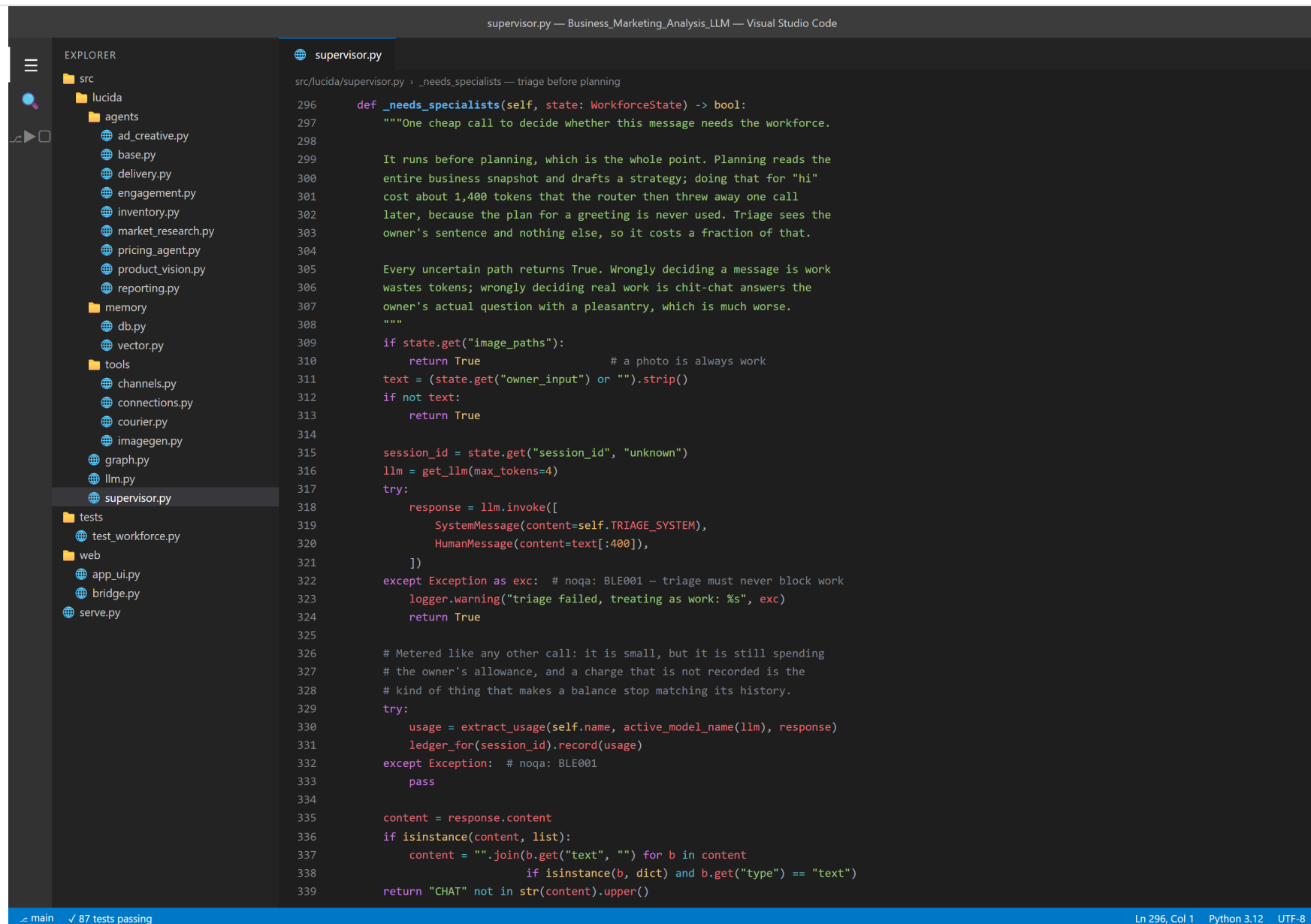


```
src/lucida/supervisor.py > ROUTER_SYSTEM — the rules the router obeys

83  ROUTER_SYSTEM = f"""You are the Supervisor of an AI workforce that runs a small business end to end.
84
85  Decide which single agent should work NEXT, or FINISH if the owner's request has been
86  fully answered.
87
88  {AGENT_CATALOG}
89
90  Rules:
91  - Choose exactly ONE agent name from the list, or FINISH.
92  - FINISH IMMEDIATELY, before any agent runs, when the message does not need
93    specialist work: a greeting, thanks, an acknowledgement, small talk, or a
94    question about you and what you can do. You will still write the reply
95    yourself, so the owner is answered – they simply do not need eight
96    specialists and a web search to be told hello. Dispatching the workforce
97    at "hi" costs the owner minutes and thousands of tokens for a sentence.
98  - Do NOT re-run an agent that has already completed successfully unless its output was an
99    error, or new information genuinely changes its inputs.
100 - Answer the question that was asked, and stop. Once the agents that were run
101   can answer it, choose FINISH – do not add adjacent work the owner did not
102   ask for. "Which product should I reorder first?" is answered by stock
103   alone; sending it on to market research, pricing and reporting turns a
104   ten-second answer into a four-minute report about something else.
105 - Route to market_research ONLY when the answer genuinely lies outside the
106   business: what rivals charge, what the market wants, what is selling
107   elsewhere. Anything answerable from the shop's own records – stock,
108   products, prices, orders, customers – must not go to web search.
109 - Respect data dependencies: pricing needs a product; ad_creative needs a price;
110   reporting is most useful once the other relevant agents have run.
111 - Never route to product_vision when no image was uploaded.
112 - If the plan is complete, or the remaining steps cannot proceed (missing owner input,
113   repeated failures), choose FINISH rather than looping.
114 - `task` is the concrete instruction that agent will receive. Be specific – it is the only
115   instruction they get from you.
116 """
117
118 # Where a run's answer should be streamed to, keyed by session. The web
```

main ✓ 87 tests passing Ln 83, Col 1 Python 3.12 UTF-8

Routing discipline written as rules: answer the question asked and stop; use web research only when the answer lies outside the business.



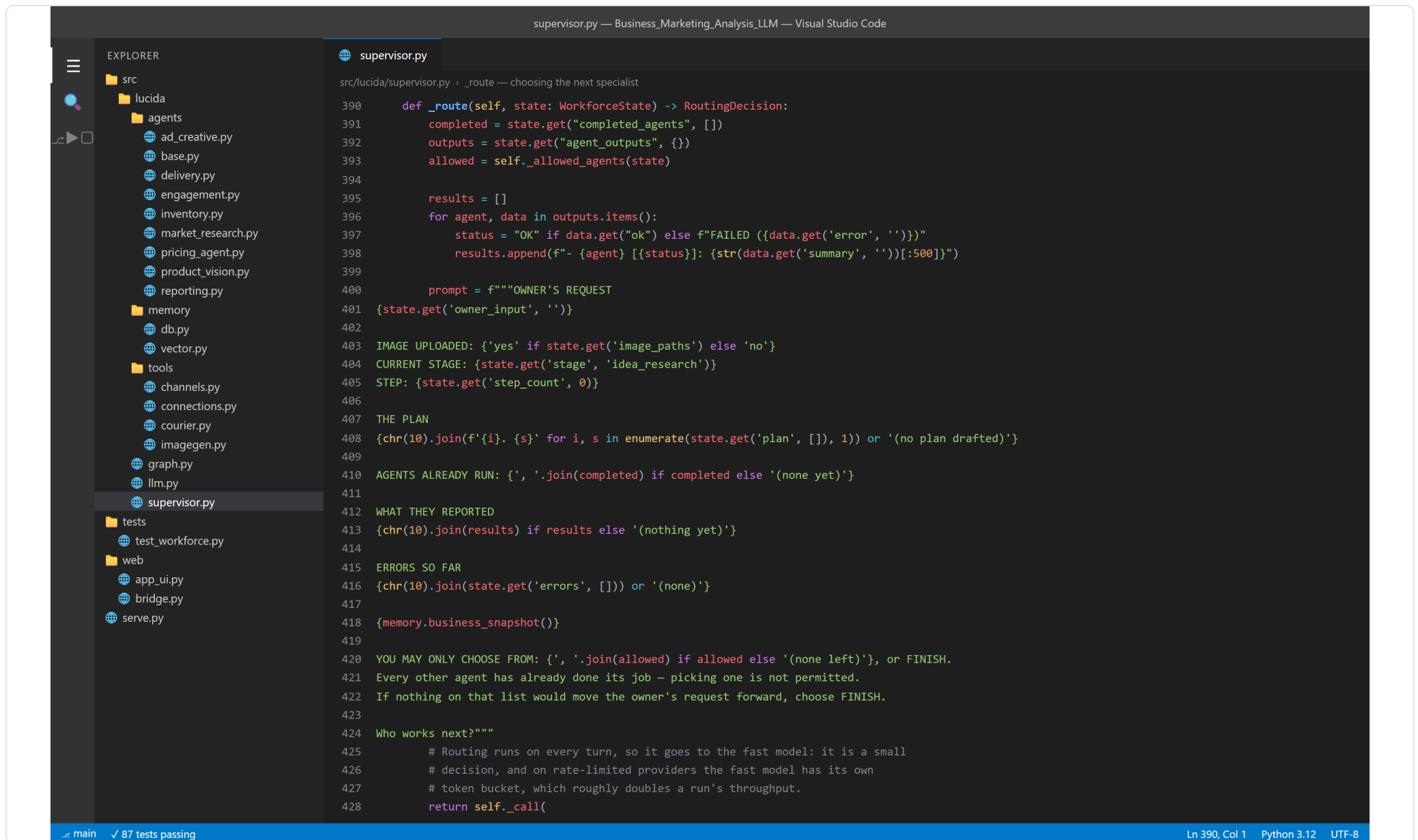
```
supervisor.py — Business_Marketing_Analysis_LLM — Visual Studio Code

src/lucida/supervisor.py > _needs_specialists — triage before planning

296 def _needs_specialists(self, state: WorkforceState) -> bool:
297     """One cheap call to decide whether this message needs the workforce.
298
299     It runs before planning, which is the whole point. Planning reads the
300     entire business snapshot and drafts a strategy; doing that for "hi"
301     cost about 1,400 tokens that the router then threw away one call
302     later, because the plan for a greeting is never used. Triage sees the
303     owner's sentence and nothing else, so it costs a fraction of that.
304
305     Every uncertain path returns True. Wrongly deciding a message is work
306     wastes tokens; wrongly deciding real work is chit-chat answers the
307     owner's actual question with a pleasantry, which is much worse.
308     """
309     if state.get("image_paths"):
310         return True # a photo is always work
311     text = (state.get("owner_input") or "").strip()
312     if not text:
313         return True
314
315     session_id = state.get("session_id", "unknown")
316     llm = get_llm(max_tokens=4)
317     try:
318         response = llm.invoke([
319             SystemMessage(content=self.TRIAGE_SYSTEM),
320             HumanMessage(content=text[:400]),
321         ])
322     except Exception as exc: # noqa: BLE001 — triage must never block work
323         logger.warning("triage failed, treating as work: %s", exc)
324         return True
325
326     # Metered like any other call: it is small, but it is still spending
327     # the owner's allowance, and a charge that is not recorded is the
328     # kind of thing that makes a balance stop matching its history.
329     try:
330         usage = extract_usage(self.name, active_model_name(llm), response)
331         ledger_for(session_id).record(usage)
332     except Exception: # noqa: BLE001
333         pass
334
335     content = response.content
336     if isinstance(content, list):
337         content = "".join(b.get("text", "") for b in content
338                             if isinstance(b, dict) and b.get("type") == "text")
339     return "CHAT" not in str(content).upper()
```

main ✓ 87 tests passing Ln 296, Col 1 Python 3.12 UTF-8

One cheap call decides whether the workforce is needed at all. It cut a greeting from 42,082 tokens to 462. Uncertainty always resolves to WORK.



supervisor.py — Business_Marketing_Analysis_LLM — Visual Studio Code

src/lucida/supervisor.py > _route — choosing the next specialist

```
390 def _route(self, state: WorkforceState) -> RoutingDecision:
391     completed = state.get("completed_agents", [])
392     outputs = state.get("agent_outputs", {})
393     allowed = self._allowed_agents(state)
394
395     results = []
396     for agent, data in outputs.items():
397         status = "OK" if data.get("ok") else f"FAILED ({data.get('error', '')})"
398         results.append(f"- {agent} [{status}]: {str(data.get('summary', ''))[:500]}")
399
400     prompt = f"""OWNER'S REQUEST
401 {state.get('owner_input', '')}
402
403 IMAGE UPLOADED: {'yes' if state.get('image_paths') else 'no'}
404 CURRENT STAGE: {state.get('stage', 'idea_research')}
405 STEP: {state.get('step_count', 0)}
406
407 THE PLAN
408 {chr(10).join(f'{i}. {s}' for i, s in enumerate(state.get('plan', []), 1)) or '(no plan drafted)'}
409
410 AGENTS ALREADY RUN: {' '.join(completed) if completed else '(none yet)'}
411
412 WHAT THEY REPORTED
413 {chr(10).join(results) if results else '(nothing yet)'}
414
415 ERRORS SO FAR
416 {chr(10).join(state.get('errors', [])) or '(none)'}
417
418 {memory.business_snapshot()}
419
420 YOU MAY ONLY CHOOSE FROM: {' '.join(allowed) if allowed else '(none left)'}, or FINISH.
421 Every other agent has already done its job – picking one is not permitted.
422 If nothing on that list would move the owner's request forward, choose FINISH.
423
424 Who works next?"""
425     # Routing runs on every turn, so it goes to the fast model: it is a small
426     # decision, and on rate-limited providers the fast model has its own
427     # token bucket, which roughly doubles a run's throughput.
428     return self._call(
```

main ✓ 87 tests passing Ln 390, Col 1 Python 3.12 UTF-8

The router is held to the allowed set in code, not only in the prompt: smaller models re-select a finished agent forever otherwise.

supervisor.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

- src
 - lucida
 - agents
 - ad_creative.py
 - base.py
 - delivery.py
 - engagement.py
 - inventory.py
 - market_research.py
 - pricing_agent.py
 - product_vision.py
 - reporting.py
 - memory
 - db.py
 - vector.py
 - tools
 - channels.py
 - connections.py
 - courier.py
 - imagegen.py
 - graph.py
 - llm.py
 - supervisor.py
 - tests
 - test_workforce.py
 - web
 - app_ui.py
 - bridge.py
 - serve.py

supervisor.py

src/lucida/supervisor.py › aggregate — composing the owner's answer

```

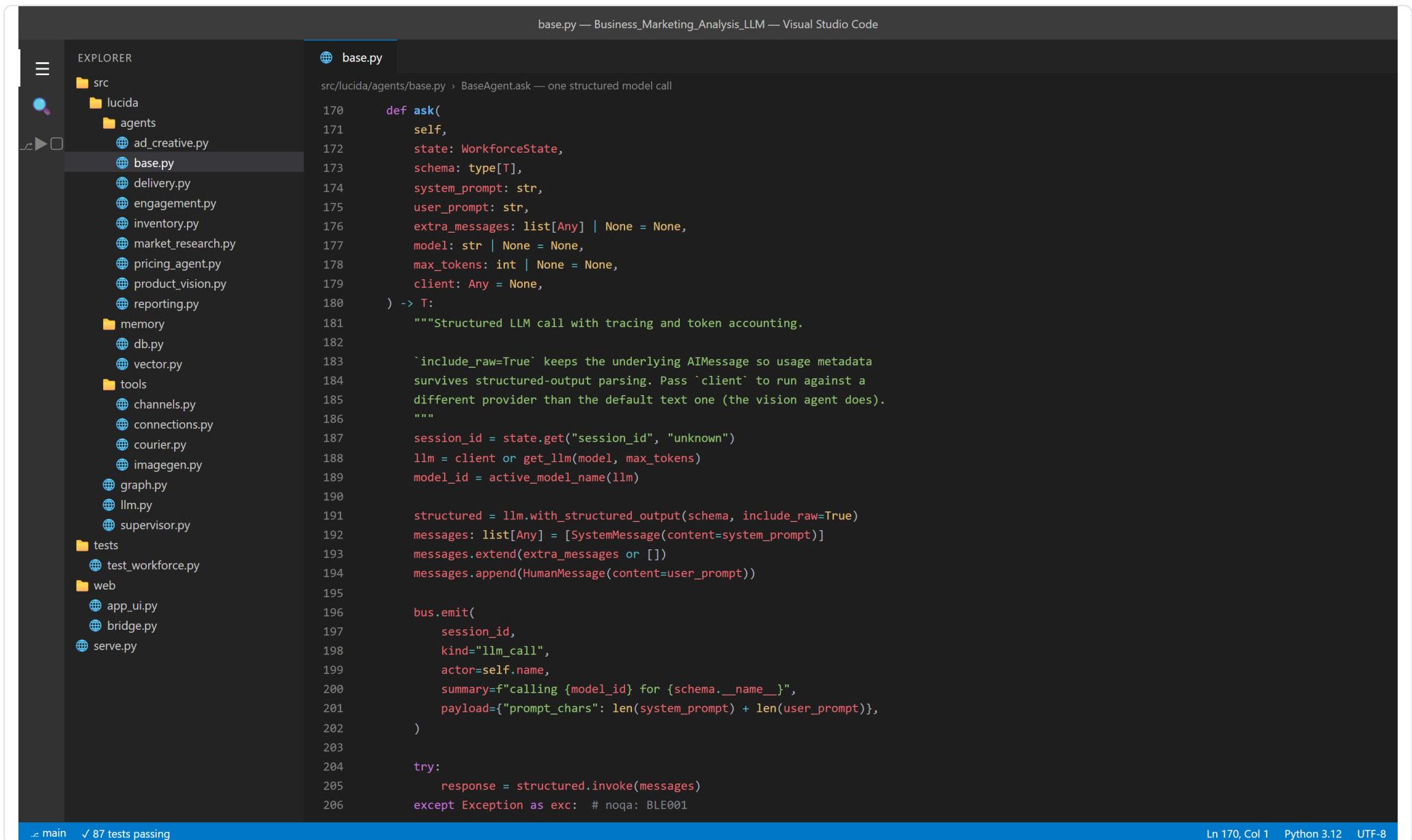
482     def aggregate(self, state: WorkforceState) -> str:
483         """Compose the owner-facing answer from everything the agents produced."""
484         outputs = state.get("agent_outputs", {})
485
486         # If Reporting ran, its Markdown already is the deliverable.
487         reporting = outputs.get("reporting", {})
488         if reporting.get("ok") and reporting.get("payload", {}).get("full_report_markdown"):
489             return reporting["payload"]["full_report_markdown"]
490
491         from .config import settings
492
493         payload_cap = 900 if settings.compact_prompts else 3000
494         # A run with no specialists is not a failure – it is the supervisor
495         # deciding the message did not need any, which is the common case for
496         # a greeting. It still gets a real, model-written reply; returning a
497         # canned "I wasn't able to complete any work" here told the owner the
498         # app was broken when it had simply answered them directly.
499         detail = []
500         for agent, data in outputs.items():
501             status = "completed" if data.get("ok") else f"FAILED: {data.get('error')}"
502             detail.append(
503                 f"### {agent} [{status}]\n{data.get('summary', '')}\n\n"
504                 f"Structured output:\n{_json(data.get('payload', {}), payload_cap)}"
505             )
506
507         # Nothing ran, so there is nothing to summarise: the owner's sentence
508         # is the entire input. Sending the stage, the approval history and a
509         # set of empty report headings invites the model to talk about them.
510         if not outputs:
511             prompt = str(state.get("owner_input", ""))[:400]
512         else:
513             prompt = f"OWNER'S ORIGINAL REQUEST\n{state.get('owner_input', '')}"
514         {state.get('owner_input', '')}
515
516         STAGE REACHED: {state.get('stage', '')}
517
518         APPROVAL DECISIONS THE OWNER MADE
519         {_json(memory.approvals(state.get('session_id', '')))}

```

main ✓ 87 tests passing

Ln 482, Col 1 Python 3.12 UTF-8

Specialist reports become one answer. A run with no specialists is not a failure; it still gets a model-written reply.



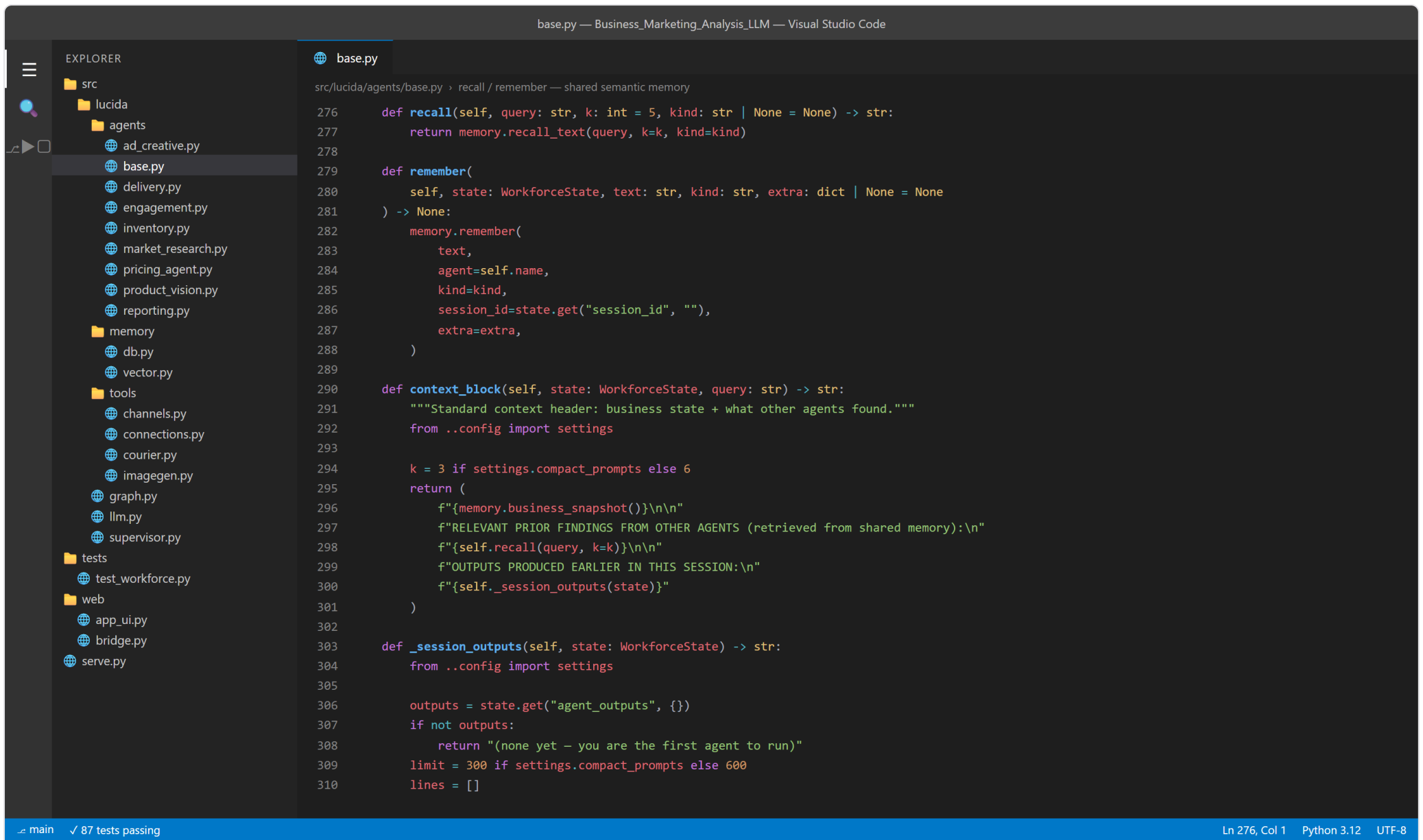
```
base.py — Business_Marketing_Analysis_LLM — Visual Studio Code

src/lucida/agents/base.py > BaseAgent.ask — one structured model call

170 def ask(
171     self,
172     state: WorkforceState,
173     schema: type[T],
174     system_prompt: str,
175     user_prompt: str,
176     extra_messages: list[Any] | None = None,
177     model: str | None = None,
178     max_tokens: int | None = None,
179     client: Any = None,
180 ) -> T:
181     """Structured LLM call with tracing and token accounting.
182
183     `include_raw=True` keeps the underlying AIMessage so usage metadata
184     survives structured-output parsing. Pass `client` to run against a
185     different provider than the default text one (the vision agent does).
186     """
187     session_id = state.get("session_id", "unknown")
188     llm = client or get_llm(model, max_tokens)
189     model_id = active_model_name(llm)
190
191     structured = llm.with_structured_output(schema, include_raw=True)
192     messages: list[Any] = [SystemMessage(content=system_prompt)]
193     messages.extend(extra_messages or [])
194     messages.append(HumanMessage(content=user_prompt))
195
196     bus.emit(
197         session_id,
198         kind="llm_call",
199         actor=self.name,
200         summary=f"calling {model_id} for {schema.__name__}",
201         payload={"prompt_chars": len(system_prompt) + len(user_prompt)},
202     )
203
204     try:
205         response = structured.invoke(messages)
206     except Exception as exc: # noqa: BLE001
```

main ✓ 87 tests passing Ln 170, Col 1 Python 3.12 UTF-8

Every agent inherits one metered, traced, schema-validated call. Nothing is parsed out of free text.



base.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

- src
 - lucida
 - agents
 - ad_creative.py
 - base.py
 - delivery.py
 - engagement.py
 - inventory.py
 - market_research.py
 - pricing_agent.py
 - product_vision.py
 - reporting.py
 - memory
 - db.py
 - vector.py
 - tools
 - channels.py
 - connections.py
 - courier.py
 - imagegen.py
 - graph.py
 - llm.py
 - supervisor.py
 - tests
 - test_workforce.py
 - web
 - app_ui.py
 - bridge.py
 - serve.py

base.py

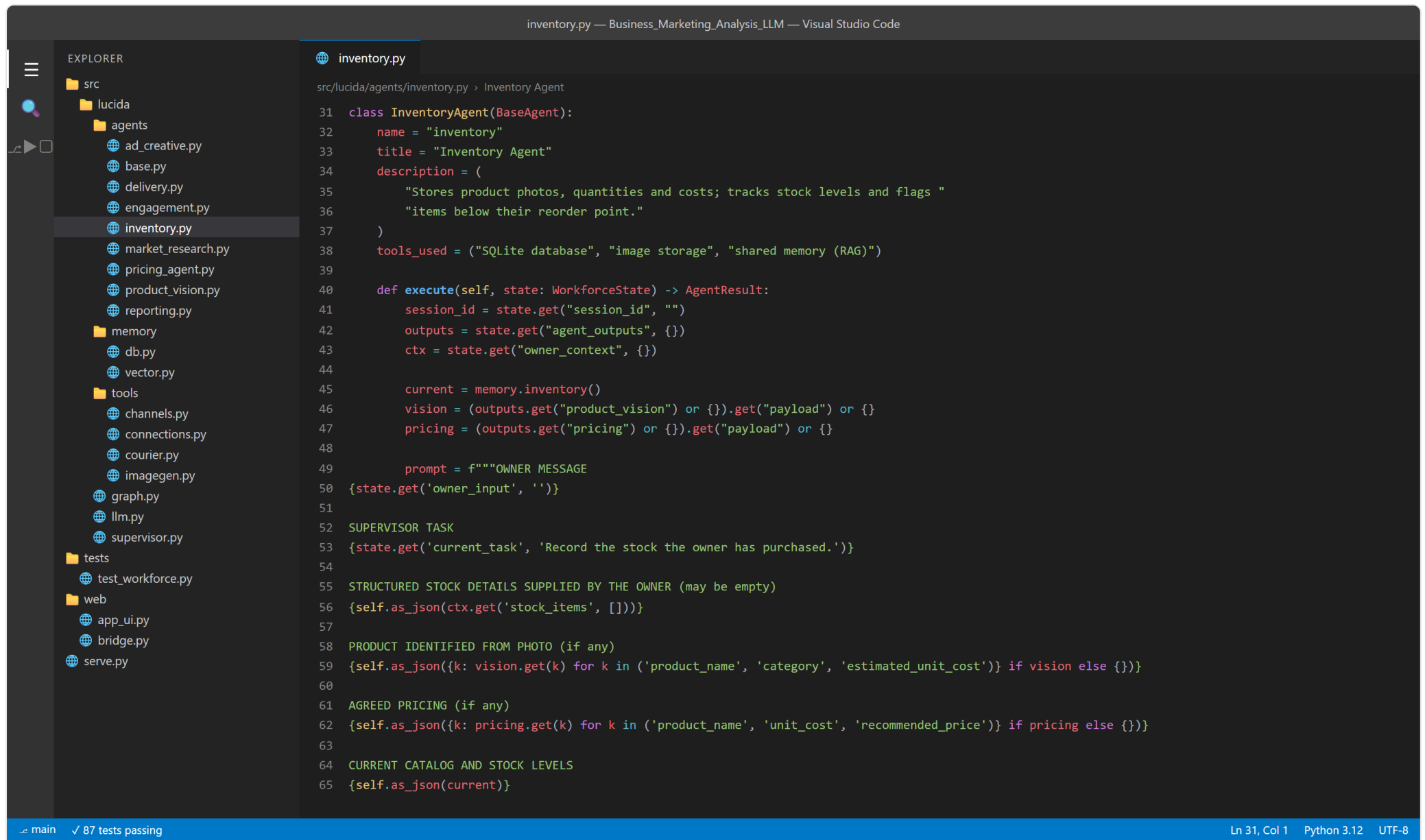
src/lucida/agents/base.py › recall / remember — shared semantic memory

```
276 def recall(self, query: str, k: int = 5, kind: str | None = None) -> str:
277     return memory.recall_text(query, k=k, kind=kind)
278
279 def remember(
280     self, state: WorkforceState, text: str, kind: str, extra: dict | None = None
281 ) -> None:
282     memory.remember(
283         text,
284         agent=self.name,
285         kind=kind,
286         session_id=state.get("session_id", ""),
287         extra=extra,
288     )
289
290 def context_block(self, state: WorkforceState, query: str) -> str:
291     """Standard context header: business state + what other agents found."""
292     from ..config import settings
293
294     k = 3 if settings.compact_prompts else 6
295     return (
296         f"{memory.business_snapshot()}\n\n"
297         f"RELEVANT PRIOR FINDINGS FROM OTHER AGENTS (retrieved from shared memory):\n"
298         f"{self.recall(query, k=k)}\n\n"
299         f"OUTPUTS PRODUCED EARLIER IN THIS SESSION:\n"
300         f"{self._session_outputs(state)}"
301     )
302
303 def _session_outputs(self, state: WorkforceState) -> str:
304     from ..config import settings
305
306     outputs = state.get("agent_outputs", {})
307     if not outputs:
308         return "(none yet – you are the first agent to run)"
309     limit = 300 if settings.compact_prompts else 600
310     lines = []
```

main ✓ 87 tests passing

Ln 276, Col 1 Python 3.12 UTF-8

What an agent learns is written back so later runs recall earlier conclusions.



```
inventory.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER
src
lucida
agents
  ad_creative.py
  base.py
  delivery.py
  engagement.py
  inventory.py
  market_research.py
  pricing_agent.py
  product_vision.py
  reporting.py
memory
  db.py
  vector.py
tools
  channels.py
  connections.py
  courier.py
  imagegen.py
  graph.py
  llm.py
  supervisor.py
tests
  test_workforce.py
web
  app_ui.py
  bridge.py
  serve.py

inventory.py
src/lucida/agents/inventory.py > Inventory Agent

31 class InventoryAgent(BaseAgent):
32     name = "inventory"
33     title = "Inventory Agent"
34     description = (
35         "Stores product photos, quantities and costs; tracks stock levels and flags "
36         "items below their reorder point."
37     )
38     tools_used = ("SQLite database", "image storage", "shared memory (RAG)")
39
40     def execute(self, state: WorkforceState) -> AgentResult:
41         session_id = state.get("session_id", "")
42         outputs = state.get("agent_outputs", {})
43         ctx = state.get("owner_context", {})
44
45         current = memory.inventory()
46         vision = (outputs.get("product_vision") or {}).get("payload") or {}
47         pricing = (outputs.get("pricing") or {}).get("payload") or {}
48
49         prompt = f"""OWNER MESSAGE
50 {state.get('owner_input', '')}
51
52 SUPERVISOR TASK
53 {state.get('current_task', 'Record the stock the owner has purchased.')}
54
55 STRUCTURED STOCK DETAILS SUPPLIED BY THE OWNER (may be empty)
56 {self.as_json(ctx.get('stock_items', []))}
57
58 PRODUCT IDENTIFIED FROM PHOTO (if any)
59 {self.as_json({k: vision.get(k) for k in ('product_name', 'category', 'estimated_unit_cost')} if vision else {})}
60
61 AGREED PRICING (if any)
62 {self.as_json({k: pricing.get(k) for k in ('product_name', 'unit_cost', 'recommended_price')} if pricing else {})}
63
64 CURRENT CATALOG AND STOCK LEVELS
65 {self.as_json(current)}
```

main ✓ 87 tests passing Ln 31, Col 1 Python 3.12 UTF-8

One of eight specialists. Quantities, reorder levels, and what runs out first.

pricing_agent.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

- src
 - lucida
 - agents
 - ad_creative.py
 - base.py
 - delivery.py
 - engagement.py
 - inventory.py
 - market_research.py
 - pricing_agent.py
 - product_vision.py
 - reporting.py
 - memory
 - db.py
 - vector.py
 - tools
 - channels.py
 - connections.py
 - courier.py
 - imagegen.py
 - graph.py
 - llm.py
 - supervisor.py
 - tests
 - test_workforce.py
 - web
 - app_ui.py
 - bridge.py
 - serve.py

pricing_agent.py

src/lucida/agents/pricing_agent.py > Pricing Agent

```

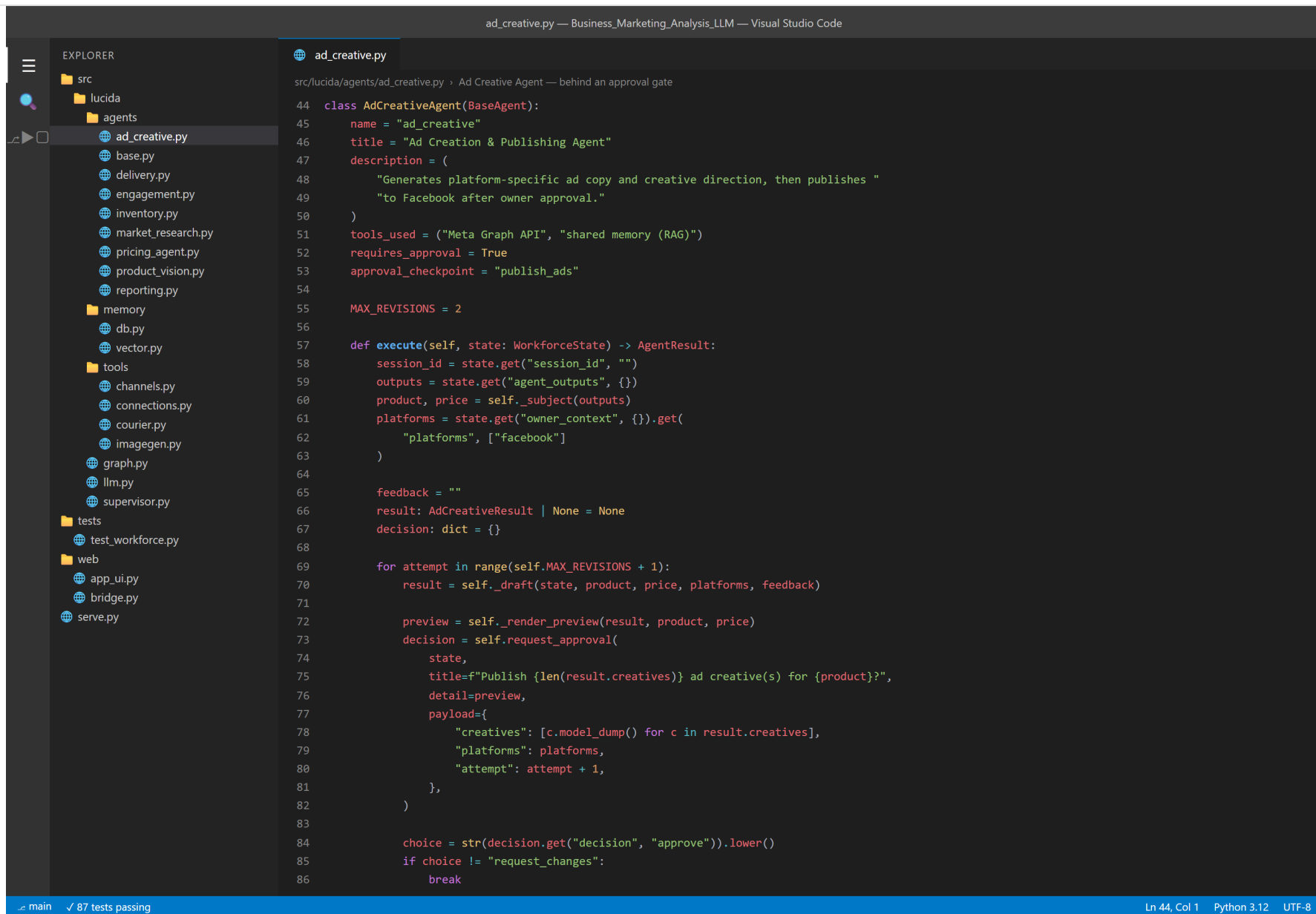
33 class PricingAgent(BaseAgent):
34     name = "pricing"
35     title = "Pricing & Cost Agent"
36     description = (
37         "Cost-plus pricing with margin, break-even and competitor comparison, "
38         "computed in a real Python sandbox."
39     )
40     tools_used = ("code execution (sandboxed Python)", "shared memory (RAG)")
41
42     def execute(self, state: WorkforceState) -> AgentResult:
43         session_id = state.get("session_id", "")
44         outputs = state.get("agent_outputs", {})
45
46         product_name, unit_cost, price_low, price_high = self._gather_inputs(state, outputs)
47         fixed_costs = float(state.get("owner_context", {}).get("fixed_costs", 0) or 0)
48         expected_units = int(
49             state.get("owner_context", {}).get("expected_monthly_units", 0) or 0
50         )
51
52         # Provisional price: mid-band if we have one, else cost-plus 60%.
53         provisional = (
54             (price_low + price_high) / 2 if price_high > 0 else unit_cost * 1.6
55         )
56
57         # Deterministic core metrics, computed in Python.
58         base = margin_analysis(unit_cost, provisional, fixed_costs, expected_units)
59         bus.emit(
60             session_id,
61             kind="tool_call",
62             actor=self.name,
63             summary=f"margin_analysis(cost={unit_cost}, price={provisional:.2f})",
64             payload=base,
65         )
66
67         # A scenario sweep, executed in the sandbox, so the model can see trade-offs.
68         sweep_code = """

```

main ✓ 87 tests passing

Ln 33, Col 1 Python 3.12 UTF-8

Unit cost, price, margin and break-even, computed rather than estimated.



```
ad_creative.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER
src
lucida
agents
  ad_creative.py
  base.py
  delivery.py
  engagement.py
  inventory.py
  market_research.py
  pricing_agent.py
  product_vision.py
  reporting.py
  memory
    db.py
    vector.py
  tools
    channels.py
    connections.py
    courier.py
    imagegen.py
    graph.py
    llm.py
    supervisor.py
  tests
    test_workforce.py
  web
    app_ui.py
    bridge.py
    serve.py

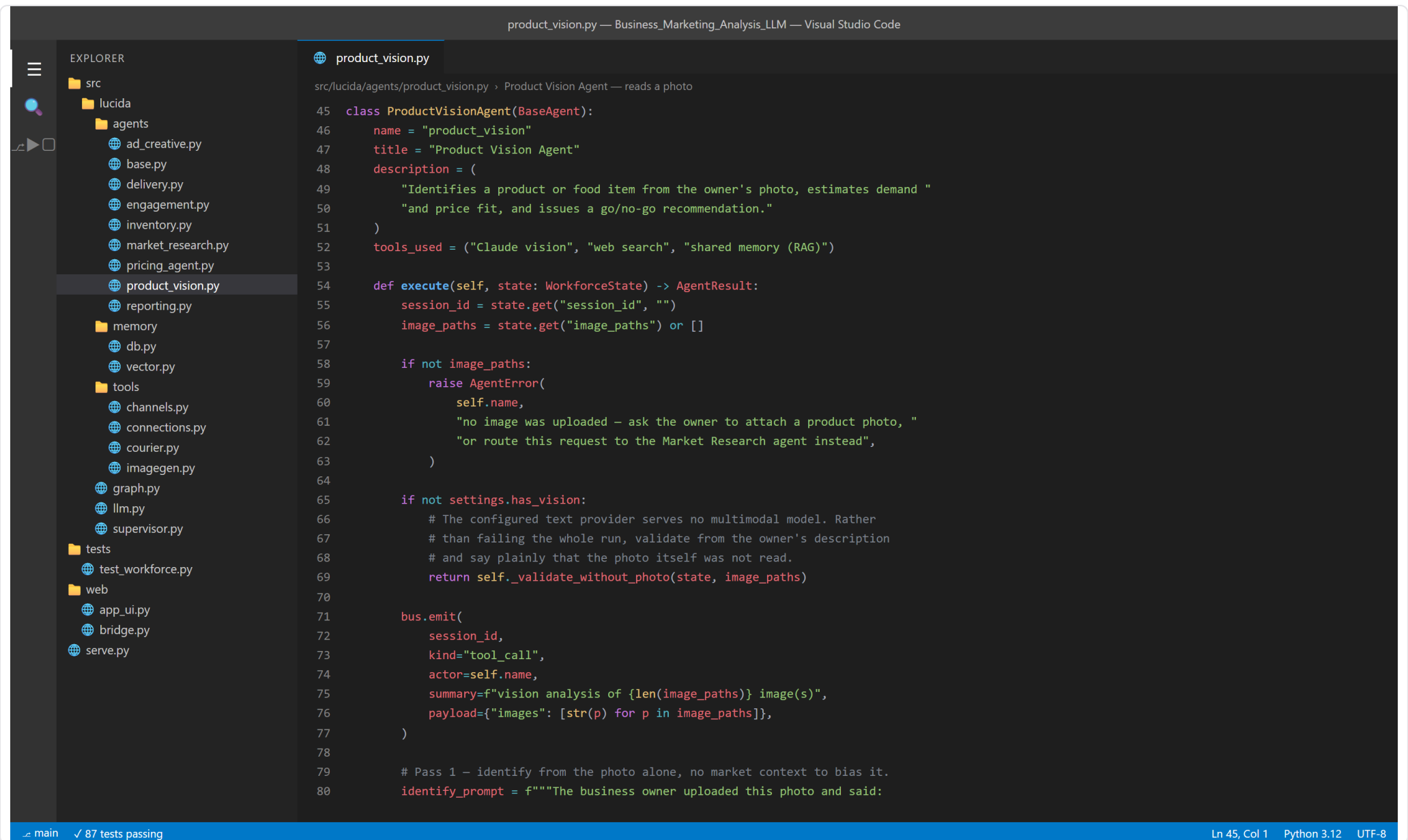
ad_creative.py
src/lucida/agents/ad_creative.py · Ad Creative Agent — behind an approval gate

44 class AdCreativeAgent(BaseAgent):
45     name = "ad_creative"
46     title = "Ad Creation & Publishing Agent"
47     description = (
48         "Generates platform-specific ad copy and creative direction, then publishes "
49         "to Facebook after owner approval."
50     )
51     tools_used = ("Meta Graph API", "shared memory (RAG)")
52     requires_approval = True
53     approval_checkpoint = "publish_ads"
54
55     MAX_REVISIONS = 2
56
57     def execute(self, state: WorkforceState) -> AgentResult:
58         session_id = state.get("session_id", "")
59         outputs = state.get("agent_outputs", {})
60         product, price = self._subject(outputs)
61         platforms = state.get("owner_context", {}).get(
62             "platforms", ["facebook"]
63         )
64
65         feedback = ""
66         result: AdCreativeResult | None = None
67         decision: dict = {}
68
69         for attempt in range(self.MAX_REVISIONS + 1):
70             result = self._draft(state, product, price, platforms, feedback)
71
72             preview = self._render_preview(result, product, price)
73             decision = self.request_approval(
74                 state,
75                 title=f"Publish {len(result.creatives)} ad creative(s) for {product}?",
76                 detail=preview,
77                 payload={
78                     "creatives": [c.model_dump() for c in result.creatives],
79                     "platforms": platforms,
80                     "attempt": attempt + 1,
81                 },
82             )
83
84             choice = str(decision.get("decision", "approve")).lower()
85             if choice != "request_changes":
86                 break
```

main ✓ 87 tests passing

Ln 44, Col 1 Python 3.12 UTF-8

requires_approval = True. Publishing suspends the graph until the owner answers.



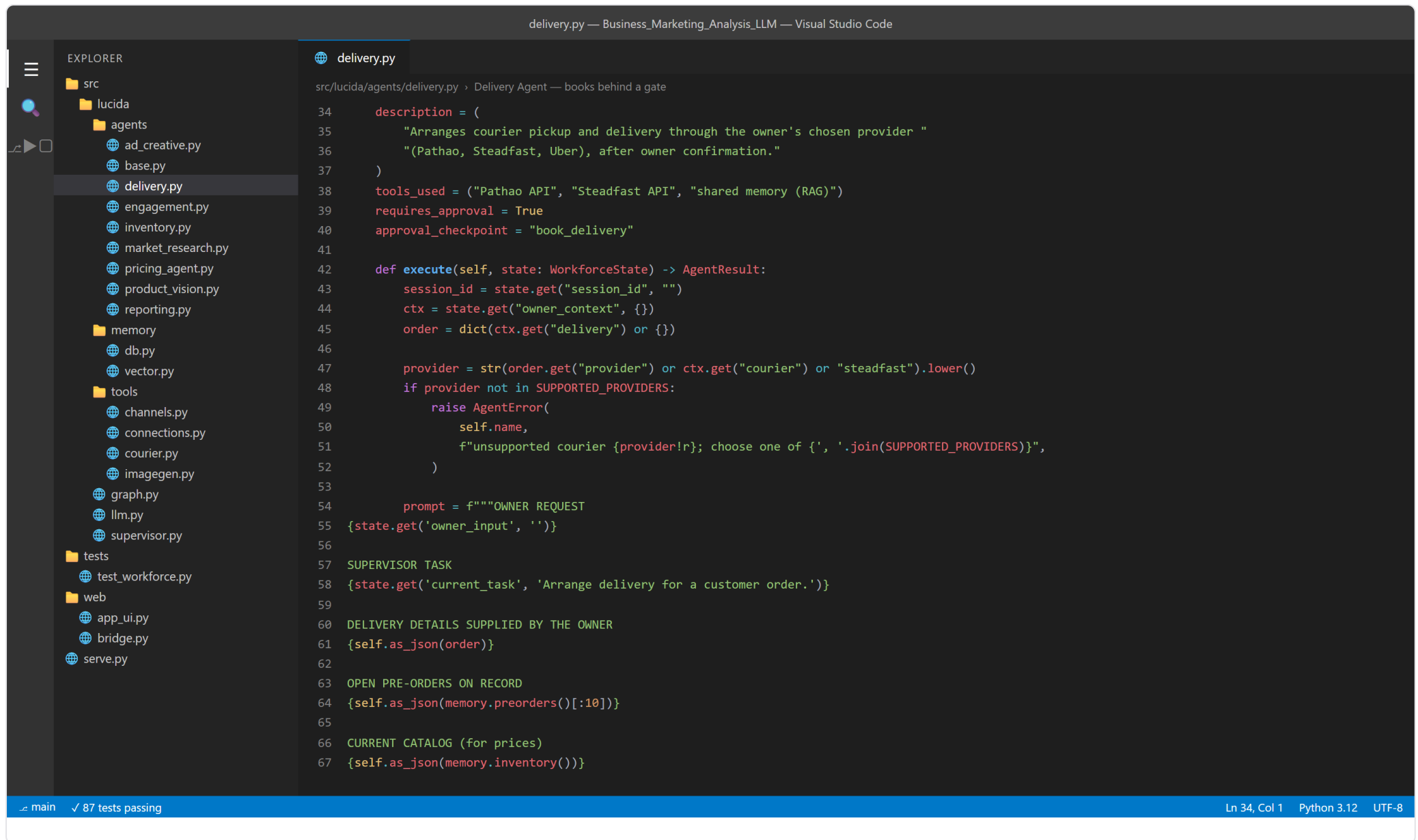
```
product_vision.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER
src
lucida
agents
  ad_creative.py
  base.py
  delivery.py
  engagement.py
  inventory.py
  market_research.py
  pricing_agent.py
  product_vision.py
  reporting.py
memory
  db.py
  vector.py
tools
  channels.py
  connections.py
  courier.py
  imagegen.py
  graph.py
  llm.py
  supervisor.py
tests
  test_workforce.py
web
  app_ui.py
  bridge.py
  serve.py

product_vision.py
src/lucida/agents/product_vision.py › Product Vision Agent — reads a photo

45 class ProductVisionAgent(BaseAgent):
46     name = "product_vision"
47     title = "Product Vision Agent"
48     description = (
49         "Identifies a product or food item from the owner's photo, estimates demand "
50         "and price fit, and issues a go/no-go recommendation."
51     )
52     tools_used = ("Claude vision", "web search", "shared memory (RAG)")
53
54     def execute(self, state: WorkforceState) -> AgentResult:
55         session_id = state.get("session_id", "")
56         image_paths = state.get("image_paths") or []
57
58         if not image_paths:
59             raise AgentError(
60                 self.name,
61                 "no image was uploaded – ask the owner to attach a product photo, "
62                 "or route this request to the Market Research agent instead",
63             )
64
65         if not settings.has_vision:
66             # The configured text provider serves no multimodal model. Rather
67             # than failing the whole run, validate from the owner's description
68             # and say plainly that the photo itself was not read.
69             return self._validate_without_photo(state, image_paths)
70
71         bus.emit(
72             session_id,
73             kind="tool_call",
74             actor=self.name,
75             summary=f"vision analysis of {len(image_paths)} image(s)",
76             payload={"images": [str(p) for p in image_paths]},
77         )
78
79         # Pass 1 – identify from the photo alone, no market context to bias it.
80         identify_prompt = f"""The business owner uploaded this photo and said:
```

The multimodal path: a photograph in, a sell-or-do-not verdict out.



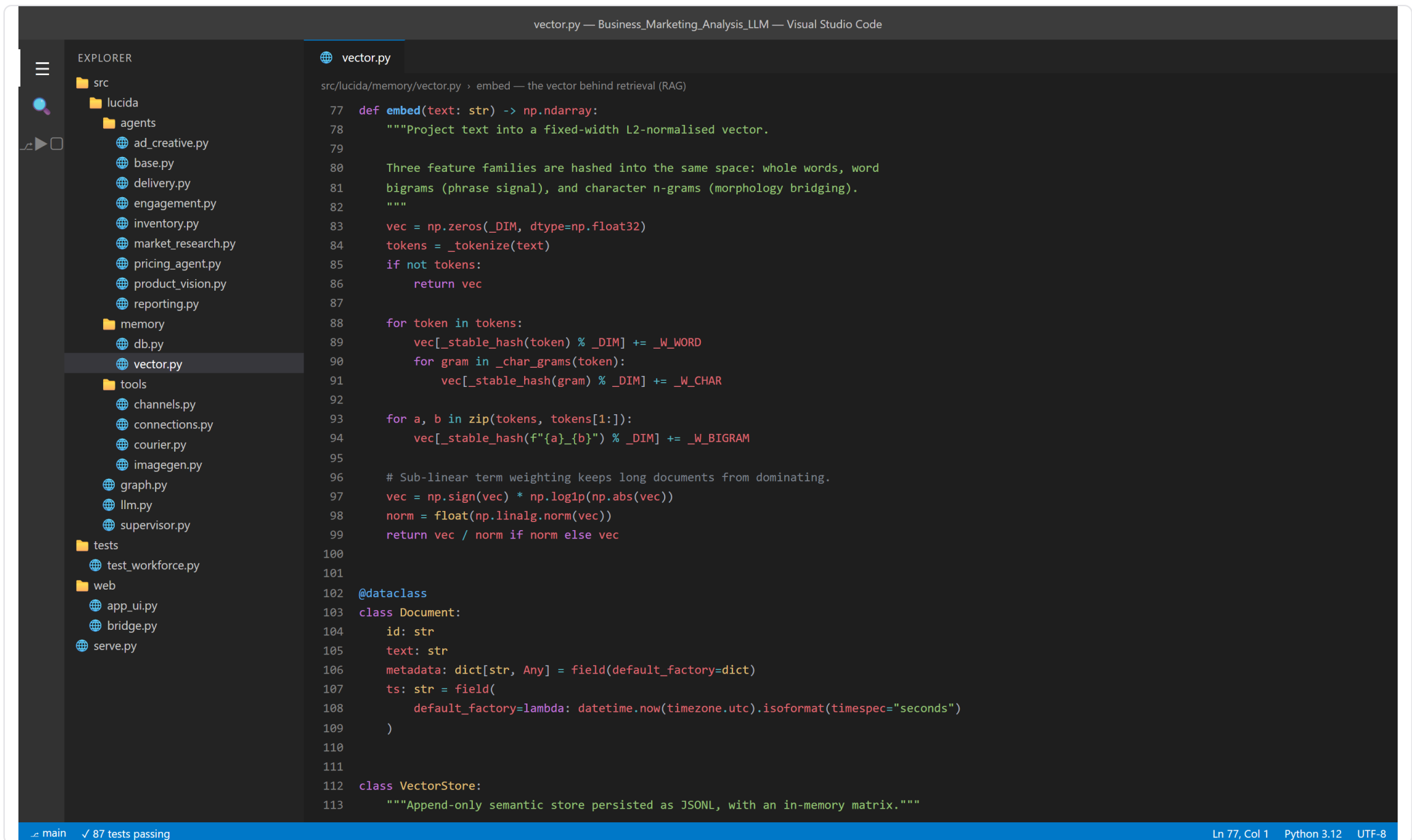
```
delivery.py — Business_Marketing_Analysis_LLM — Visual Studio Code

src/lucida/agents/delivery.py › Delivery Agent — books behind a gate

34     description = (
35         "Arranges courier pickup and delivery through the owner's chosen provider "
36         "(Pathao, Steadfast, Uber), after owner confirmation."
37     )
38     tools_used = ("Pathao API", "Steadfast API", "shared memory (RAG)")
39     requires_approval = True
40     approval_checkpoint = "book_delivery"
41
42     def execute(self, state: WorkforceState) -> AgentResult:
43         session_id = state.get("session_id", "")
44         ctx = state.get("owner_context", {})
45         order = dict(ctx.get("delivery") or {})
46
47         provider = str(order.get("provider") or ctx.get("courier") or "steadfast").lower()
48         if provider not in SUPPORTED_PROVIDERS:
49             raise AgentError(
50                 self.name,
51                 f"unsupported courier {provider!r}; choose one of {'', '.join(SUPPORTED_PROVIDERS)}",
52             )
53
54         prompt = f"""OWNER REQUEST
55 {state.get('owner_input', '')}
56
57 SUPERVISOR TASK
58 {state.get('current_task', 'Arrange delivery for a customer order.')}
59
60 DELIVERY DETAILS SUPPLIED BY THE OWNER
61 {self.as_json(order)}
62
63 OPEN PRE-ORDERS ON RECORD
64 {self.as_json(memory.preorders()[:10])}
65
66 CURRENT CATALOG (for prices)
67 {self.as_json(memory.inventory())}
```

main ✓ 87 tests passing Ln 34, Col 1 Python 3.12 UTF-8

Booking a courier spends money, so it is gated the same way publishing is.



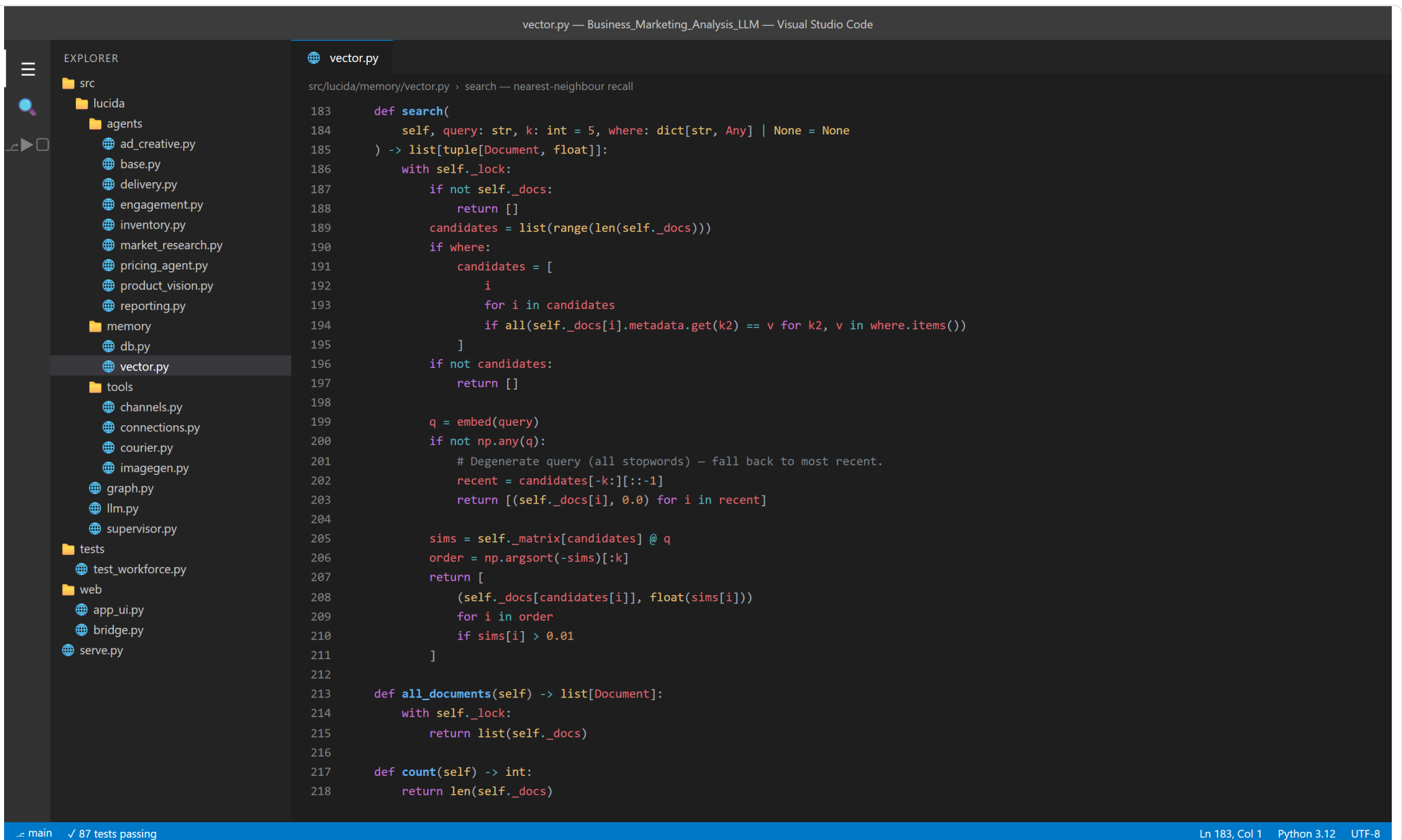
```
vector.py — Business_Marketing_Analysis_LLM — Visual Studio Code

src/lucida/memory/vector.py > embed — the vector behind retrieval (RAG)

77 def embed(text: str) -> np.ndarray:
78     """Project text into a fixed-width L2-normalised vector.
79
80     Three feature families are hashed into the same space: whole words, word
81     bigrams (phrase signal), and character n-grams (morphology bridging).
82     """
83     vec = np.zeros(_DIM, dtype=np.float32)
84     tokens = _tokenize(text)
85     if not tokens:
86         return vec
87
88     for token in tokens:
89         vec[_stable_hash(token) % _DIM] += _W_WORD
90         for gram in _char_grams(token):
91             vec[_stable_hash(gram) % _DIM] += _W_CHAR
92
93     for a, b in zip(tokens, tokens[1:]):
94         vec[_stable_hash(f"{a}_{b}") % _DIM] += _W_BIGRAM
95
96     # Sub-linear term weighting keeps long documents from dominating.
97     vec = np.sign(vec) * np.log1p(np.abs(vec))
98     norm = float(np.linalg.norm(vec))
99     return vec / norm if norm else vec
100
101
102 @dataclass
103 class Document:
104     id: str
105     text: str
106     metadata: dict[str, Any] = field(default_factory=dict)
107     ts: str = field(
108         default_factory=lambda: datetime.now(timezone.utc).isoformat(timespec="seconds")
109     )
110
111
112 class VectorStore:
113     """Append-only semantic store persisted as JSONL, with an in-memory matrix."""
```

main ✓ 87 tests passing Ln 77, Col 1 Python 3.12 UTF-8

The embedding used for semantic recall across runs.



```
vector.py — Business_Marketing_Analysis_LLM — Visual Studio Code

src/lucida/memory/vector.py · search — nearest-neighbour recall

183     def search(
184         self, query: str, k: int = 5, where: dict[str, Any] | None = None
185     ) -> list[tuple[Document, float]]:
186         with self._lock:
187             if not self._docs:
188                 return []
189             candidates = list(range(len(self._docs)))
190             if where:
191                 candidates = [
192                     i
193                     for i in candidates
194                     if all(self._docs[i].metadata.get(k2) == v for k2, v in where.items())
195                 ]
196             if not candidates:
197                 return []
198
199             q = embed(query)
200             if not np.any(q):
201                 # Degenerate query (all stopwords) – fall back to most recent.
202                 recent = candidates[-k:][::-1]
203                 return [(self._docs[i], 0.0) for i in recent]
204
205             sims = self._matrix[candidates] @ q
206             order = np.argsort(-sims)[:k]
207             return [
208                 (self._docs[candidates[i]], float(sims[i]))
209                 for i in order
210                 if sims[i] > 0.01
211             ]
212
213     def all_documents(self) -> list[Document]:
214         with self._lock:
215             return list(self._docs)
216
217     def count(self) -> int:
218         return len(self._docs)
```

main ✓ 87 tests passing Ln 183, Col 1 Python 3.12 UTF-8

Cosine similarity over stored knowledge; this is the retrieval half of RAG.

db.py — Business_Marketing_Analysis_LLM — Visual Studio Code

src/lucida/memory/db.py > The per-shop schema

```

24 CREATE TABLE IF NOT EXISTS business_profile (
25     id            INTEGER PRIMARY KEY CHECK (id = 1),
26     owner_name    TEXT,
27     business_name TEXT,
28     niche         TEXT,
29     location      TEXT,
30     currency      TEXT,
31     monthly_budget REAL,
32     notes         TEXT,
33     updated_at    TEXT
34 );
35
36 CREATE TABLE IF NOT EXISTS products (
37     id            INTEGER PRIMARY KEY AUTOINCREMENT,
38     name          TEXT NOT NULL,
39     category      TEXT,
40     description   TEXT,
41     unit_cost     REAL,
42     sell_price    REAL,
43     photo_path    TEXT,
44     source_agent  TEXT,
45     created_at    TEXT,
46     UNIQUE(name)
47 );
48
49 CREATE TABLE IF NOT EXISTS inventory (
50     id            INTEGER PRIMARY KEY AUTOINCREMENT,
51     product_id    INTEGER NOT NULL REFERENCES products(id),
52     quantity      INTEGER NOT NULL DEFAULT 0,
53     reorder_level INTEGER NOT NULL DEFAULT 5,
54     updated_at    TEXT
55 );
56
57 CREATE TABLE IF NOT EXISTS stock_movements (
58     id            INTEGER PRIMARY KEY AUTOINCREMENT,
59     product_id    INTEGER NOT NULL REFERENCES products(id),
60     delta         INTEGER NOT NULL,

```

main ✓ 87 tests passing Ln 24, Col 1 Python 3.12 UTF-8

22 tables. Each shop gets its own file, so isolation does not depend on a WHERE clause.

llm.py — Business_Marketing_Analysis_LLM — Visual Studio Code

```

src/lucida/llm.py > _Failover — moving on when a model cannot answer
313 class _Failover:
314     """A client that moves to the next model when one cannot answer.
315
316     Wraps rather than subclasses because each provider returns its own client
317     type, and the only thing every caller uses is `.invoke`.
318     """
319
320     def __init__(self, models: list[tuple[str, str]], max_tokens: int) -> None:
321         self._models = models
322         self._max_tokens = max_tokens
323
324     def _run(self, call):
325         """Walk the chain until something answers.
326
327         Plain and structured calls take the same path, because the reasons a
328         model cannot answer – capped, retired, busy, or refusing to emit a
329         tool call – are the same either way, and the structured calls are
330         most of the agents' work.
331         """
332         last: Exception | None = None
333         for i, (provider, model) in enumerate(self._models):
334             for attempt in (1, 2):
335                 try:
336                     return call(build_client(provider, model, self._max_tokens))
337                 except Exception as exc: # noqa: BLE001
338                     if not should_try_next(exc):
339                         raise
340                     last = exc
341                     # A rejected tool call is intermittent: the same model and
342                     # the same prompt succeed on a second pass. Spending one
343                     # retry here is cheaper than giving up on a model that
344                     # works, which on a short chain means giving up entirely.
345                     if attempt == 1 and is_flaky_tool_call(exc):
346                         logger.warning(
347                             "%s did not answer as a tool call; trying it again",
348                             model)
349                         continue
350                     break
351             if i + 1 < len(self._models):
352                 logger.warning("%s %s; trying %s", model, _why(last),

```

main ✓ 87 tests passing Ln 313, Col 1 Python 3.12 UTF-8

Capped, retired, or refusing to emit a tool call all advance the chain. Free tiers vanish without warning; this survives it.

connections.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

- src
 - lucida
 - agents
 - ad_creative.py
 - base.py
 - delivery.py
 - engagement.py
 - inventory.py
 - market_research.py
 - pricing_agent.py
 - product_vision.py
 - reporting.py
 - memory
 - db.py
 - vector.py
 - tools
 - channels.py
 - connections.py
 - courier.py
 - imagegen.py
 - graph.py
 - llm.py
 - supervisor.py
 - tests
 - test_workforce.py
 - web
 - app_ui.py
 - bridge.py
 - serve.py

connections.py

src/lucida/tools/connections.py > pathao_token — a real external API

```

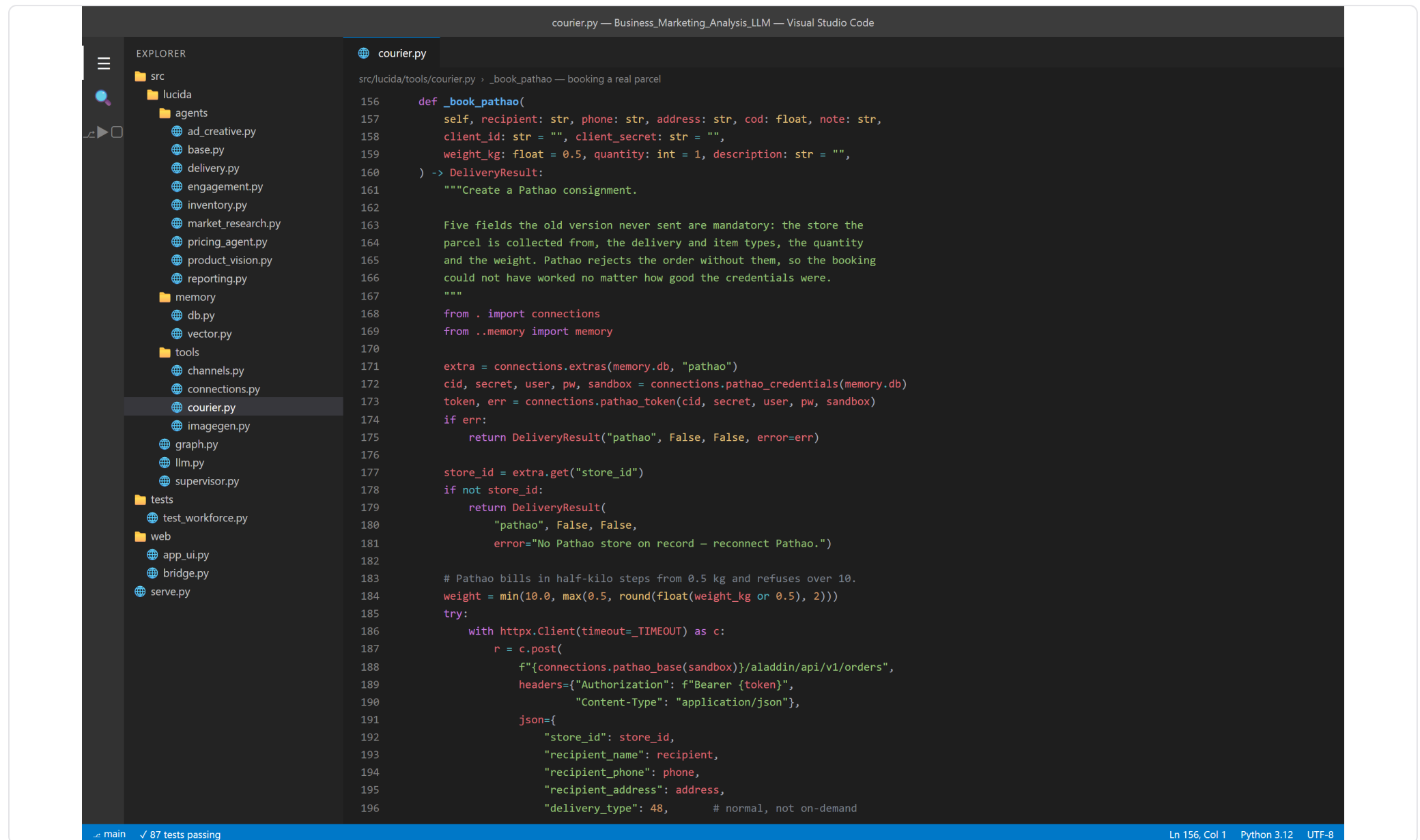
330 def pathao_token(client_id: str, client_secret: str, username: str,
331                   password: str, sandbox: bool = False) -> tuple[str, str]:
332     """A Pathao access token, or ('', why not).
333
334     `grant_type=password`, not client_credentials: the client pair identifies
335     the integration, the login identifies the merchant, and Pathao wants
336     both. Sending only the pair returns "The user credentials were incorrect",
337     which reads like a wrong key and is really a missing login.
338     """
339     try:
340         with httpx.Client(timeout=TIMEOUT) as c:
341             r = c.post(f"{pathao_base(sandbox)}/aladdin/api/v1/issue-token",
342                      json={"client_id": client_id,
343                          "client_secret": client_secret,
344                          "grant_type": "password",
345                          "username": username, "password": password})
346         if r.status_code != 200:
347             detail = ""
348             try:
349                 detail = r.json().get("message") or ""
350             except Exception: # noqa: BLE001
351                 detail = r.text[:120]
352             return "", f"Pathao refused those credentials: {detail}"
353         token = r.json().get("access_token") or ""
354         return (token, "") if token else ("", "Pathao returned no token.")
355     except Exception as exc: # noqa: BLE001
356         return "", str(exc)
357
358
359 def pathao_get(path: str, token: str, sandbox: bool = False) -> tuple[dict, str]:
360     try:
361         with httpx.Client(timeout=TIMEOUT) as c:
362             r = c.get(f"{pathao_base(sandbox)}/{path}",
363                     headers={"Authorization": f"Bearer {token}",
364                             "Content-Type": "application/json"})
365             if r.status_code != 200:
366                 return {}, f"Pathao returned {r.status_code}"

```

main ✓ 87 tests passing

Ln 330, Col 1 Python 3.12 UTF-8

Pathao issues tokens by password grant. The client pair alone is refused — a bug found and fixed against the live API.



courier.py — Business_Marketing_Analysis_LLM — Visual Studio Code

src/lucida/tools/courier.py > _book_pathao — booking a real parcel

```
156 def _book_pathao(
157     self, recipient: str, phone: str, address: str, cod: float, note: str,
158     client_id: str = "", client_secret: str = "",
159     weight_kg: float = 0.5, quantity: int = 1, description: str = "",
160 ) -> DeliveryResult:
161     """Create a Pathao consignment.
162
163     Five fields the old version never sent are mandatory: the store the
164     parcel is collected from, the delivery and item types, the quantity
165     and the weight. Pathao rejects the order without them, so the booking
166     could not have worked no matter how good the credentials were.
167     """
168     from . import connections
169     from ..memory import memory
170
171     extra = connections.extras(memory.db, "pathao")
172     cid, secret, user, pw, sandbox = connections.pathao_credentials(memory.db)
173     token, err = connections.pathao_token(cid, secret, user, pw, sandbox)
174     if err:
175         return DeliveryResult("pathao", False, False, error=err)
176
177     store_id = extra.get("store_id")
178     if not store_id:
179         return DeliveryResult(
180             "pathao", False, False,
181             error="No Pathao store on record - reconnect Pathao.")
182
183     # Pathao bills in half-kilo steps from 0.5 kg and refuses over 10.
184     weight = min(10.0, max(0.5, round(float(weight_kg or 0.5), 2)))
185     try:
186         with httpx.Client(timeout=_TIMEOUT) as c:
187             r = c.post(
188                 f"{connections.pathao_base(sandbox)}/aladdin/api/v1/orders",
189                 headers={"Authorization": f"Bearer {token}",
190                        "Content-Type": "application/json"},
191                 json={
192                     "store_id": store_id,
193                     "recipient_name": recipient,
194                     "recipient_phone": phone,
195                     "recipient_address": address,
196                     "delivery_type": 48, # normal, not on-demand
```

main ✓ 87 tests passing Ln 156, Col 1 Python 3.12 UTF-8

A real consignment, with store id, weight and item counts, against Pathao's own endpoint.

channels.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

```

src
├── lucida
│   └── agents
│       ├── ad_creative.py
│       ├── base.py
│       ├── delivery.py
│       ├── engagement.py
│       ├── inventory.py
│       ├── market_research.py
│       ├── pricing_agent.py
│       ├── product_vision.py
│       └── reporting.py
│   ├── memory
│   │   ├── db.py
│   │   └── vector.py
│   └── tools
│       ├── channels.py
│       ├── connections.py
│       ├── courier.py
│       ├── imagegen.py
│       ├── graph.py
│       ├── llm.py
│       └── supervisor.py
├── tests
│   └── test_workforce.py
└── web
    ├── app_ui.py
    ├── bridge.py
    └── serve.py

```

channels.py

src/lucida/tools/channels.py > read_telegram — the live customer channel

```

617 def read_telegram(limit: int = 25, db=None) -> Inbox:
618     """Customer messages sent to the bot, and the buttons they tapped.
619
620     Telegram keeps only a short retention window for old updates, so we advance
621     a stored `last_update_id` after each sync. That prevents replaying the same
622     message and allows the next message in the conversation to be processed.
623     """
624     if not telegram_ready(db):
625         return Inbox([], simulated=True)
626
627     # The caller's database, not the global singleton: the poll loop runs
628     # with no signed-in owner, and storing the offset somewhere other than
629     # where the messages go means it never advances.
630     db = db if db is not None else _shop_db()
631     last_seen = _telegram_offset(db)
632     params = {"limit": max(1, min(limit, 100)), "timeout": 0}
633     if last_seen > 0:
634         params["offset"] = last_seen + 1
635
636     data, err = _telegram_get("getUpdates", params)
637     if err:
638         logger.warning("telegram read failed: %s", err)
639         return Inbox([], simulated=False, error=err)
640
641     out: list[dict[str, Any]] = []
642     max_update_id = last_seen
643     for update in data.get("result", []):
644         update_id = int(update.get("update_id") or 0)
645         if update_id > max_update_id:
646             max_update_id = update_id
647
648     # A tapped button arrives as a callback_query, not a message. It is
649     # the customer answering a question the shop asked, so it enters the
650     # inbox as what the tap meant – "/rate 5" – and travels the same path
651     # as anything they could have typed instead.
652     callback = update.get("callback_query")

```

main ✓ 87 tests passing

Ln 617, Col 1 Python 3.12 UTF-8

Per-shop bot tokens, an advancing update offset, and messages written to that shop alone.

```

serve.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER
src
  lucida
    agents
      ad_creative.py
      base.py
      delivery.py
      engagement.py
      inventory.py
      market_research.py
      pricing_agent.py
      product_vision.py
      reporting.py
    memory
      db.py
      vector.py
    tools
      channels.py
      connections.py
      courier.py
      imagegen.py
      graph.py
      llm.py
      supervisor.py
  tests
    test_workforce.py
  web
    app_ui.py
    bridge.py
    serve.py

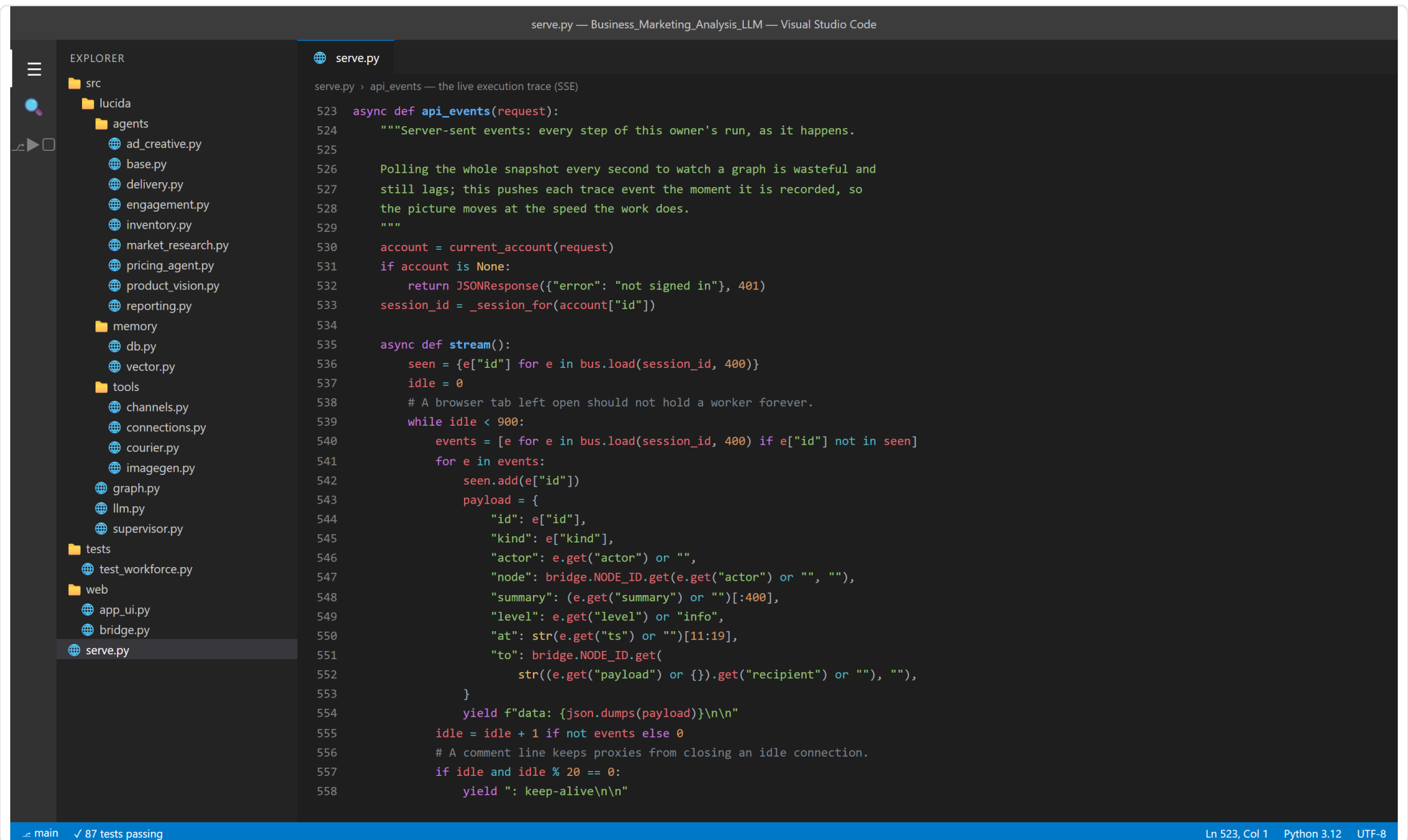
serve.py
serve.py > api_delivery_book — the gate, server-side

283 async def api_delivery_book(request):
284     """Hand the parcel to the courier the shop connected."""
285     account = current_account(request)
286     if account is None:
287         return JSONResponse({"error": "not signed in"}, 401)
288     body = await request.json()
289
290     provider = connections.courier_ready(memory.db)
291     if not provider:
292         return JSONResponse({"error": (
293             "No courier connected. Add Steadfast or Pathao on the Connect "
294             "screen and the booking goes through your own account.")}, 400)
295
296     # Booking is the one irreversible thing on this screen: a real courier is
297     # sent to a real address and the shop is charged. The confirmation is
298     # enforced here rather than only in the browser, so a stray click, a
299     # replayed request or another client cannot book by accident. The first
300     # call returns what is about to happen; only a call carrying `confirm`
301     # actually books.
302     if not bool(body.get("confirm")):
303         return JSONResponse({
304             "needs_confirmation": True,
305             "provider": provider,
306             "sandbox": bool(connections.extras(memory.db, "pathao").get("sandbox"))
307                 if provider == "pathao" else False,
308             "summary": {
309                 "customer": str(body.get("customer") or ""),
310                 "phone": str(body.get("phone") or ""),
311                 "address": str(body.get("address") or ""),
312                 "product": str(body.get("product") or ""),
313                 "cod_amount": float(body.get("cod_amount") or 0),
314             },
315         })
316
317     result = await asyncio.to_thread(
318         courier.book,

```

main ✓ 87 tests passing Ln 283, Col 1 Python 3.12 UTF-8

The confirmation is enforced on the server, so a stray click or a replayed request cannot book.



serve.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

- src
 - lucida
 - agents
 - ad_creative.py
 - base.py
 - delivery.py
 - engagement.py
 - inventory.py
 - market_research.py
 - pricing_agent.py
 - product_vision.py
 - reporting.py
 - memory
 - db.py
 - vector.py
 - tools
 - channels.py
 - connections.py
 - courier.py
 - imagegen.py
 - graph.py
 - llm.py
 - supervisor.py
 - tests
 - test_workforce.py
 - web
 - app_ui.py
 - bridge.py
 - serve.py

serve.py

```

523 async def api_events(request):
524     """Server-sent events: every step of this owner's run, as it happens.
525
526     Polling the whole snapshot every second to watch a graph is wasteful and
527     still lags; this pushes each trace event the moment it is recorded, so
528     the picture moves at the speed the work does.
529     """
530     account = current_account(request)
531     if account is None:
532         return JSONResponse({"error": "not signed in"}, 401)
533     session_id = _session_for(account["id"])
534
535     async def stream():
536         seen = {e["id"] for e in bus.load(session_id, 400)}
537         idle = 0
538         # A browser tab left open should not hold a worker forever.
539         while idle < 900:
540             events = [e for e in bus.load(session_id, 400) if e["id"] not in seen]
541             for e in events:
542                 seen.add(e["id"])
543                 payload = {
544                     "id": e["id"],
545                     "kind": e["kind"],
546                     "actor": e.get("actor") or "",
547                     "node": bridge.NODE_ID.get(e.get("actor") or "", ""),
548                     "summary": (e.get("summary") or "")[:400],
549                     "level": e.get("level") or "info",
550                     "at": str(e.get("ts") or "")[11:19],
551                     "to": bridge.NODE_ID.get(
552                         str((e.get("payload") or {}).get("recipient") or ""), ""),
553                     )
554                 }
555                 yield f"data: {json.dumps(payload)}\n\n"
556             idle = idle + 1 if not events else 0
557             # A comment line keeps proxies from closing an idle connection.
558             if idle and idle % 20 == 0:
559                 yield ": keep-alive\n\n"

```

Ln 523, Col 1 Python 3.12 UTF-8

One held-open connection pushes each step as it is recorded, scoped to one owner's session.

```

bridge.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER
src
├── lucida
│   └── agents
│       ├── ad_creative.py
│       ├── base.py
│       ├── delivery.py
│       ├── engagement.py
│       ├── inventory.py
│       ├── market_research.py
│       ├── pricing_agent.py
│       ├── product_vision.py
│       └── reporting.py
│   ├── memory
│   │   ├── db.py
│   │   └── vector.py
│   ├── tools
│   │   ├── channels.py
│   │   ├── connections.py
│   │   ├── courier.py
│   │   ├── imagegen.py
│   │   ├── graph.py
│   │   ├── llm.py
│   │   └── supervisor.py
│   ├── tests
│   │   └── test_workforce.py
│   └── web
│       ├── app_ui.py
│       ├── bridge.py
│       └── serve.py

web/bridge.py > cost_bars — what the dashboard renders

437 def cost_bars(session_id: str) -> list[dict]:
438     """Where the tokens went, per agent.
439
440     Shape must match the design's COST_BARS: {name, tok, cost, pct, color}.
441     `pct` is a bar width scaled against the heaviest agent.
442
443     Read from the durable `llm_usage` trace rather than the in-process
444     ledger: `ledger_for()` lives in a module-level dict, so a restart – or
445     simply the other front-end – sees nothing. The trace survives both.
446
447     (The design's separate COST_ROWS is an *ingredient* cost breakdown –
448     filling, dough, oil, packaging. Lucida records no per-ingredient costing,
449     so that constant is deliberately left un-overridden.)
450     """
451     per: dict[str, dict[str, float]] = {}
452
453     for e in bus.load(session_id, 600):
454         if e["kind"] != "llm_usage":
455             continue
456         row = per.setdefault(str(e["actor"]), {"tokens": 0.0, "usd": 0.0})
457         row["usd"] += float((e.get("payload") or {}).get("cost_usd") or 0.0)
458         m = _USAGE_RE.search(e.get("summary") or "")
459         if m:
460             row["tokens"] += int(m.group(1).replace(",", ""))
461             row["tokens"] += int(m.group(2).replace(",", ""))
462
463     # Fall back to the live ledger for the session running in this process,
464     # which has data the trace has not been asked for yet.
465     if not per:
466         for call in ledger_for(session_id).calls:
467             row = per.setdefault(call.agent, {"tokens": 0.0, "usd": 0.0})
468             row["tokens"] += call.total_tokens
469             row["usd"] += call.cost_usd
470
471     if not per:
472         return []

```

main ✓ 87 tests passing Ln 437, Col 1 Python 3.12 UTF-8

Ranked and scaled by tokens, which are measured, rather than by a price that was not.

test_workforce.py — Business_Marketing_Analysis_LLM — Visual Studio Code

EXPLORER

- src
 - lucida
 - agents
 - ad_creative.py
 - base.py
 - delivery.py
 - engagement.py
 - inventory.py
 - market_research.py
 - pricing_agent.py
 - product_vision.py
 - reporting.py
 - memory
 - db.py
 - vector.py
 - tools
 - channels.py
 - connections.py
 - courier.py
 - imagegen.py
 - graph.py
 - llm.py
 - supervisor.py
 - tests
 - test_workforce.py
 - web
 - app_ui.py
 - bridge.py
 - serve.py

test_workforce.py

tests/test_workforce.py · One of 87 tests

```
186 def test_telegram_offset_advances_and_persists(monkeypatch):
187     from lucida.tools import channels
188
189     class DummyDB:
190         def __init__(self):
191             self._state = {"last_update_id": 10}
192
193         def query(self, sql, params=()):
194             if "SELECT last_update_id FROM telegram_sync_state" in sql:
195                 return [{"last_update_id": self._state["last_update_id"]}]}
196             return []
197
198         def execute(self, sql, params=()):
199             if "INSERT INTO telegram_sync_state" in sql or "UPDATE telegram_sync_state" in sql:
200                 self._state["last_update_id"] = params[0]
201                 return 1
202             return 0
203
204     db = DummyDB()
205     monkeypatch.setattr(channels, "_shop_db", lambda: db)
206
207     calls = {}
208
209     def fake_get(method, params):
210         calls[method] = params.copy()
211         return {
212             "ok": True,
213             "result": [{
214                 "update_id": 11,
215                 "message": {
216                     "message_id": 77,
217                     "date": 1720000000,
218                     "chat": {"id": 123},
219                     "from": {"id": 123, "first_name": "A"},
220                     "text": "hello",
```

main ✓ 87 tests passing Ln 186, Col 1 Python 3.12 UTF-8

The suite runs with no API key and passes from a clean extract.